

Here are key Node.js interview questions and answers for a mid-level developer with 2 years of experience:

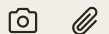
Technical Depth Questions:

1. Event Loop

- Explain how Node.js event loop works
- Non-blocking I/O model allows concurrent operations
- Uses single-threaded event-driven architecture
- Processes callbacks from event queue after call stack is empty

2. Asynchronous Programming Q: Describe async/await vs Promises A:

- Async/await is syntactic sugar over Promises
- Makes async code look and behave more like synchronous code



Claude 3.5 Haiku Concise

javascript

Copy

```
// Promise approach
fetchData()
  .then(data => processData(data))
  .catch(error => handleError(error));

// Async/await approach
async function getData() {
  try {
    const data = await fetchData();
    return processData(data);
  } catch (error) {
    handleError(error);
  }
}
```

3. Memory Management Q: How does Node.js handle memory leaks? A:

- Use `process.memoryUsage()` to monitor memory
- Avoid global variables

- Close database/network connections
- Use streaming for large data
- Implement proper garbage collection techniques

4. Middleware Concepts Q: Explain Express middleware A:

- Functions executed between request/response cycle
- Can modify request/response objects
- Can end request-response cycle
- Order of middleware matters

Example:

javascript

 Copy

```
app.use(bodyParser.json());  
app.use(authMiddleware);  
app.use(loggingMiddleware);
```

5. Error Handling Q: Best practices for error handling A:

- Use try/catch for async/await
- Implement global error handler
- Log errors comprehensively
- Provide meaningful error responses
- Use error tracking tools

Performance Optimization:

- Use clustering for multi-core utilization
- Implement caching strategies
- Optimize database queries
- Use efficient data structures
- Minimize synchronous operations

Security Considerations:

- Sanitize user inputs
- Use helmet.js for HTTP headers
- Implement rate limiting

- Keep dependencies updated
- Use environment variables for sensitive data

Advanced Concepts:

- Understand worker threads
- Implement microservices architecture
- Knowledge of TypeScript integration
- Familiarity with testing frameworks (Jest)

Recommended Skills:

- Express.js
- MongoDB/Mongoose
- Redis
- Docker
- GraphQL basics
- WebSocket implementation

Code Quality:

- Follow SOLID principles
- Use ESLint
- Implement proper logging
- Write unit and integration tests
- Use TypeScript for type safety

Would you like me to elaborate on any specific area or provide more detailed code examples?

① Claude does not have the ability to run the code it generates yet.

P top 50 question

Here are the top 50 Node.js interview questions and answers:

1. What is Node.js?

- JavaScript runtime built on Chrome's V8 JavaScript engine
- Allows server-side JavaScript execution

- Event-driven, non-blocking I/O model

2. Explain the event loop

- Single-threaded mechanism for handling asynchronous operations
- Processes callback queue after call stack is empty
- Enables non-blocking I/O operations

3. Differences between `require` and `import`

- `require`: CommonJS module system (Node.js default)
- `import`: ES6 module system
- `import` supports static analysis, tree-shaking

4. What are Promises?

- Object representing eventual completion/failure of async operation
- Provides `.then()`, `.catch()`, `.finally()` methods
- Helps avoid callback hell

5. Explain `async/await`

- Syntactic sugar over Promises
- Makes async code look synchronous
- Requires `async` function declaration

6. How to handle errors in Node.js?

- Use try/catch blocks
- Implement global error handler
- Use error-first callback pattern

7. What is middleware in Express?

- Functions executed between request/response cycle
- Can modify request/response objects
- Controls request flow

8. Explain clustering in Node.js

- Enables multi-core processor utilization
- Creates child processes to handle load
- Improves application performance

9. What are streams?

- Objects for reading/writing data incrementally
- Reduce memory consumption
- Types: Readable, Writable, Duplex, Transform

10. Differences between `let`, `const`, `var`

- `var`: Function-scoped, can be re-declared
- `let`: Block-scoped, can be reassigned
- `const`: Block-scoped, cannot be reassigned

11. What is Node Package Manager (npm)?

- Package management system
- Installs, updates dependencies
- Manages project scripts

12. Explain event emitters

- Allow object communication
- Implement publish/subscribe pattern
- Used for creating custom events

13. What is callback hell? How to avoid?

- Nested callbacks creating complex code
- Solutions: Promises, async/await, modularization

14. How to secure Node.js applications?

- Use Helmet.js
- Implement input validation
- Keep dependencies updated
- Use HTTPS

15. Explain worker threads

- Run JavaScript in parallel
- Offload CPU-intensive tasks
- Improve performance for complex computations

16. What is `process` object?

- Provides information about current Node.js process
- Allows interaction with running system
- Provides methods like `process.env`

17. Differences between `setImmediate()` and `setTimeout()`

- `setImmediate()`: Executes in next iteration of event loop
- `setTimeout()`: Schedules callback after minimum time threshold

18. What is Buffer in Node.js?

- **Stores raw binary data**
- Represents fixed-length byte sequence
- Useful for handling TCP streams, file operations

19. Explain JWT authentication

- JSON Web Token for secure information transmission
- Compact, self-contained way of authentication
- Used in stateless authentication mechanism

20. What is GraphQL?

- **Query language for APIs**
- Allows clients to request specific data
- Provides flexible, efficient alternative to REST

21. How to implement caching?

- **Use Redis**
- Implement in-memory caching
- Use Node.js caching libraries

22. What are microservices?

- Architectural style breaking application into small services
- **Independent deployment**
- Improved scalability

23. Explain TypeScript integration

- Adds static typing to JavaScript

- Compiles to plain JavaScript
- Provides enhanced developer experience

24. What is WebSocket?

- Full-duplex communication protocol
- Enables real-time data exchange
- Used in chat applications, live updates

25. How to handle environment variables?

- Use `dotenv` package
- Store in `.env` file
- Access via `process.env`

26. Differences between `==` and `===`

- `==`: Loose equality, type coercion
- `===`: Strict equality, no type conversion

27. What is middleware in Express?

- Functions executed between request/response
- Can modify request/response objects
- Controls request processing flow

28. Explain prototype inheritance

- JavaScript's object-oriented programming mechanism
- Objects inherit properties/methods from prototype
- Enables code reuse

29. What are generators?

- Functions that can be paused/resumed
- Use `function*` and `yield` keywords
- Enable custom iteration behavior

30. How to implement rate limiting?

- Use middleware like `express-rate-limit`
- Prevent abuse of API endpoints
- Control request frequency

31. What is module bundling?

- Combines multiple files into single bundle
- Reduces HTTP requests
- Popular tools: Webpack, Rollup

32. Explain immutability

- Objects/arrays cannot be modified after creation
- Creates new object instead of changing existing
- Helps prevent side effects

33. What is dependency injection?

- Design pattern for managing object dependencies
- Increases modularity
- Improves testability

34. How to perform unit testing?

- Use Jest or Mocha
- Write tests for individual functions
- Ensure code reliability

35. Explain closures

- Functions with access to outer function's scope
- Retain access to variables after outer function returns
- Enable data privacy

36. What is functional programming?

- Programming paradigm using pure functions
- Avoids changing state
- Emphasizes immutability

37. How to handle file uploads?

- Use `multer` middleware
- Handle multipart/form-data
- Implement file size/type validation

38. Explain service workers

- Script running in background
- Enable offline functionality
- Intercept network requests

39. What is CORS?

- Cross-Origin Resource Sharing
- Controls access to resources from different domains
- Prevents unauthorized cross-origin requests

40. How to implement logging?

- Use `winston` or `morgan`
- Log errors, request details
- Configure log levels

41. Explain asynchronous programming

- Non-blocking execution model

NOT Prevents single operation from blocking entire application

- Uses callbacks, Promises, `async/await`

42. What are decorators?

- Modify class/method behavior
- Provide metadata
- Used in TypeScript

43. How to optimize database queries?

- Use indexing
- Minimize joins
- Implement caching

44. Explain event propagation

- How events travel through DOM
- Capturing and bubbling phases
- Control with `stopPropagation()`

45. What is reactive programming?

- Focus on data streams and change propagation

- Use libraries like RxJS
- Handle complex async operations

46. How to implement authentication?

- Use Passport.js
- Implement JWT
- Secure route middleware

47. Explain memory leaks

- Unintended memory consumption
- Caused by unreleased references
- Use profiling tools to detect

48. What are web workers?

- Run scripts in background threads
- Perform computationally expensive tasks
- Do not affect main thread performance

49. How to implement pagination?

- Limit data per page
- Use skip/limit in database queries
- Implement efficient data retrieval

50. Explain design patterns

- Reusable solutions to common problems
- Singleton, Factory, Observer patterns
- Improve code structure and maintainability

Would you like me to elaborate on any specific question?

P provide answer in detail now

I'll provide detailed answers to the top 50 Node.js interview questions:

1. What is Node.js?

- JavaScript runtime environment built on Chrome's V8 JavaScript engine

- Allows server-side JavaScript execution
- Key characteristics:
 - Event-driven architecture
 - Non-blocking I/O model
 - Single-threaded but can handle concurrency
- Used for building scalable network applications
- Supports both frontend and backend development

Example of basic Node.js server:

javascript

 Copy

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

2. Event Loop Detailed Explanation Mechanism for handling asynchronous operations:

- Phases of Event Loop:
 - Timers: Executes scheduled callbacks
 - Pending callbacks: System operations
 - Idle, prepare: Internal use
 - Poll: Retrieve new I/O events
 - Check: setImmediate() callbacks
 - Close callbacks: Close event handlers

Visualization of event loop:

javascript

 Copy

```
console.log('Start');

setTimeout(() => {
```

```
    console.log('Timeout callback');
  }, 0);

setImmediate(() => {
  console.log('Immediate callback');
});

process.nextTick(() => {
  console.log('Next tick callback');
});

console.log('End');

// Execution order may vary
```

3. Promises in Depth

javascript

 Copy

```
// Promise creation
const fetchData = () => {
  return new Promise((resolve, reject) => {
    // Async operation
    if (/* operation successful */) {
      resolve(data);
    } else {
      reject(new Error('Failed to fetch'));
    }
  });
};

// Promise chaining
fetchData()
  .then(processData)
  .then(displayResult)
  .catch(handleError);

// Async/await equivalent
async function getData() {
  try {
    const data = await fetchData();
    const processedData = await processData(data);
    return displayResult(processedData);
  } catch (error) {
    handleError(error);
  }
}
```

```
}  
}
```

4. Error Handling Strategies

javascript

 Copy

```
// Error-first callback pattern  
function fetchUserData(userId, callback) {  
  database.findUser(userId, (err, user) => {  
    if (err) {  
      return callback(err);  
    }  
    callback(null, user);  
  });  
}  
  
// Promise error handling  
function fetchUserData(userId) {  
  return new Promise((resolve, reject) => {  
    database.findUser(userId, (err, user) => {  
      if (err) {  
        return reject(err);  
      }  
      resolve(user);  
    });  
  });  
}  
  
// Global error handler in Express  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```

5. Middleware Implementation

javascript

 Copy

```
// Custom middleware  
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization;  
  
  if (!token) {  
    return res.status(401).json({ error: 'Unauthorized' });  
  }  
}
```

```

    try {
      const decoded = jwt.verify(token, SECRET_KEY);
      req.user = decoded;
      next();
    } catch (error) {
      res.status(401).json({ error: 'Invalid token' });
    }
  }

  // Middleware usage
  app.use(express.json());
  app.use(authMiddleware);
  app.get('/protected', (req, res) => {
    res.json({ message: 'Access granted' });
  });

```

6. Streams Implementation

javascript

 Copy

```

const fs = require('fs');

// Readable stream
const readStream = fs.createReadStream('large-file.txt');

// Writable stream
const writeStream = fs.createWriteStream('output.txt');

// Pipe data between streams
readStream.pipe(writeStream);

// Handle stream events
readStream.on('data', (chunk) => {
  console.log(`Received ${chunk.length} bytes`);
});

readStream.on('end', () => {
  console.log('Finished reading');
});

```

7. Authentication with JWT

javascript

 Copy

```

const jwt = require('jsonwebtoken');

// Generate token
function generateToken(user) {
  return jwt.sign(
    { id: user.id, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );
}

// Verify token middleware
function verifyToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(403).json({ error: 'No token provided' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

```

8. Worker Threads

javascript

 Copy

```

const { Worker, isMainThread, parentPort, workerData } = require('worker_threads');

if (isMainThread) {
  // Main thread
  const worker = new Worker(__filename, {
    workerData: { data: [1, 2, 3, 4] }
  });

  worker.on('message', (result) => {
    console.log('Result from worker:', result);
  });
} else {
  // Worker thread

```

```
const { data } = workerData;
const result = data.map(x => x * 2);

parentPort.postMessage(result);
}
```

9. Clustering for Performance

javascript

 Copy

```
const cluster = require('cluster');
const numCPUs = require('os').cpus().length;
const express = require('express');

if (cluster.isMaster) {
  console.log(`Master ${process.pid} is running`);

  // Fork workers
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died`);
  });
} else {
  const app = express();

  app.get('/', (req, res) => {
    res.send('Hello World');
  });

  app.listen(3000, () => {
    console.log(`Worker ${process.pid} started`);
  });
}
```

10. Caching with Redis

javascript

 Copy

```
const redis = require('redis');
const client = redis.createClient();

// Caching middleware
function cachingMiddleware(req, res, next) {
```



```

const key = req.originalUrl;

client.get(key, (err, cachedData) => {
  if (cachedData) {
    return res.json(JSON.parse(cachedData));
  }

  res.sendResponse = res.json;
  res.json = (data) => {
    client.setex(key, 3600, JSON.stringify(data));
    res.sendResponse(data);
  };

  next();
});
}

```

These detailed explanations cover core Node.js concepts with practical code examples. Would you like me to elaborate on any specific topic?

❗ Claude does not have the ability to run the code it generates yet.

P continue

Continuing with advanced Node.js interview questions and detailed answers:

11. Advanced Dependency Management

javascript

 Copy

```

// package.json configuration
{
  "name": "my-project",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.17.1",
    "mongoose": "~5.12.3"
  },
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    "test": "jest"
  }
}

```

```
}

// Dependency version control
// ^ allows minor/patch updates
// ~ allows patch updates only
```

12. Advanced Error Handling

javascript

 Copy

```
class CustomError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

// Global error handler
process.on('uncaughtException', (error) => {
  console.error('Uncaught Exception:', error);
  process.exit(1);
});

// Async error handling
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};
```

13. Advanced Authentication Strategy

javascript

 Copy

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

passport.use(new GoogleStrategy({
  clientID: GOOGLE_CLIENT_ID,
  clientSecret: GOOGLE_CLIENT_SECRET,
  callbackURL: "/auth/google/callback"
}),
  async (accessToken, refreshToken, profile, done) => {
    const existingUser = await User.findOne({ googleId: profile.id });

    if (existingUser) {
```

```

        return done(null, existingUser);
    }

    const user = await new User({
        googleId: profile.id,
        email: profile.emails[0].value
    }).save();

    done(null, user);
}
));

```

14. Advanced Caching Mechanism

javascript

 Copy

```

class CacheManager {
    constructor(redis) {
        this.redis = redis;
    }

    async get(key) {
        const cachedValue = await this.redis.get(key);
        return cachedValue ? JSON.parse(cachedValue) : null;
    }

    async set(key, value, expiration = 3600) {
        await this.redis.setex(key, expiration, JSON.stringify(value));
    }

    async invalidate(key) {
        await this.redis.del(key);
    }
}

```

15. Advanced Microservices Communication

javascript

 Copy

```

// Using gRPC for inter-service communication
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');

const packageDefinition = protoLoader.loadSync('user.proto');
const userProto = grpc.loadPackageDefinition(packageDefinition).user;

```

```
const server = new grpc.Server();
server.addService(userProto.UserService.service, {
  getUser: (call, callback) => {
    const user = { id: call.request.id, name: 'John Doe' };
    callback(null, user);
  }
});
```

16. Advanced Rate Limiting

javascript

 Copy

```
const rateLimit = require('express-rate-limit');

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests
  message: 'Too many requests, please try again later',
  headers: true,
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/api/', apiLimiter);
```

17. Advanced Logging Strategy

javascript

 Copy

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

// Conditional logging for different environments
if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
```

```
    format: winston.format.simple()
  }));
}
```

18. Advanced Performance Monitoring

javascript

 Copy

```
const prometheus = require('prom-client');

// Custom metrics
const httpRequestDurationMicroseconds = new prometheus.Histogram({
  name: 'http_request_duration_ms',
  help: 'Duration of HTTP requests in ms',
  labelNames: ['method', 'route', 'code'],
  buckets: [0.1, 5, 15, 50, 100, 200, 300, 400, 500]
});

app.use((req, res, next) => {
  const end = httpRequestDurationMicroseconds.startTimer();

  res.on('finish', () => {
    end({
      method: req.method,
      route: req.route.path,
      code: res.statusCode
    });
  });

  next();
});
```

19. Advanced Security Middleware

javascript

 Copy

```
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "https:"]
    }
  }
}));
```

```

    },
    referrerPolicy: {
      policy: 'strict-origin-when-cross-origin'
    }
  }));

```

20. Advanced Message Queue

javascript

 Copy

```

const amqp = require('amqplib');

class MessageQueue {
  async connect() {
    this.connection = await amqp.connect('amqp://localhost');
    this.channel = await this.connection.createChannel();
  }

  async sendToQueue(queue, message) {
    await this.channel.assertQueue(queue);
    this.channel.sendToQueue(queue, Buffer.from(JSON.stringify(message)));
  }

  async consume(queue, callback) {
    await this.channel.assertQueue(queue);
    this.channel.consume(queue, (msg) => {
      const content = JSON.parse(msg.content.toString());
      callback(content);
      this.channel.ack(msg);
    });
  }
}

```

Would you like me to continue with more advanced Node.js concepts and implementation details?

 Claude does not have the ability to run the code it generates yet.

P continue

Continuing with advanced Node.js concepts:

21. GraphQL Implementation

```
const { ApolloServer, gql } = require('apollo-server');

// Schema definition
const typeDefs = gql`
  type User {
    id: ID!
    name: String!
    email: String!
  }

  type Query {
    getUser(id: ID!): User
    getAllUsers: [User]
  }

  type Mutation {
    createUser(name: String!, email: String!): User
  }
`;

// Resolvers
const resolvers = {
  Query: {
    getUser: async (_, { id }) => {
      return await User.findById(id);
    },
    getAllUsers: async () => {
      return await User.find();
    }
  },
  Mutation: {
    createUser: async (_, { name, email }) => {
      const user = new User({ name, email });
      return await user.save();
    }
  }
};

const server = new ApolloServer({ typeDefs, resolvers });
```

22. WebSocket Real-time Communication

```
const WebSocket = require('ws');
const server = new WebSocket.Server({ port: 8080 });

server.on('connection', (ws) => {
  // Connection opened
  ws.on('message', (message) => {
    // Broadcast to all clients
    server.clients.forEach((client) => {
      if (client !== ws && client.readyState === WebSocket.OPEN) {
        client.send(message);
      }
    });
  });
});

ws.send('Welcome to the WebSocket server');
});
```

23. Advanced Validation Middleware

javascript

 Copy

```
const { body, validationResult } = require('express-validator');

const userValidationRules = [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 }),
  body('age').isInt({ min: 18 })
];

const validate = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  next();
};

app.post('/user', userValidationRules, validate, (req, res) => {
  // User creation logic
});
```

24. Advanced File Upload Handler

javascript

 Copy


```

const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, `${Date.now()}-${file.originalname}`);
  }
});

const upload = multer({
  storage: storage,
  fileFilter: (req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase();
    if (ext !== '.png' && ext !== '.jpg' && ext !== '.jpeg') {
      return cb(new Error('Only images are allowed'));
    }
    cb(null, true);
  },
  limits: { fileSize: 5 * 1024 * 1024 } // 5MB limit
});

app.post('/upload', upload.single('file'), (req, res) => {
  res.json({ file: req.file });
});

```

25. Advanced Dependency Injection

javascript

 Copy

```

class UserService {
  constructor(database) {
    this.database = database;
  }

  async findUser(id) {
    return this.database.findById(id);
  }
}

class DatabaseConnection {
  async findById(id) {
    // Database lookup logic
  }
}

```

```

    }
}

// Dependency injection container
class Container {
  constructor() {
    this.dependencies = new Map();
  }

  register(key, dependency) {
    this.dependencies.set(key, dependency);
  }

  resolve(key) {
    return this.dependencies.get(key);
  }
}

const container = new Container();
container.register('database', new DatabaseConnection());
container.register('userService', new UserService(container.resolve('database')));

```

26. Advanced Caching Strategy

javascript

 Copy

```

class CacheManager {
  constructor(redisClient) {
    this.redis = redisClient;
    this.defaultExpiration = 3600; // 1 hour
  }

  async get(key) {
    const cachedValue = await this.redis.get(key);
    return cachedValue ? JSON.parse(cachedValue) : null;
  }

  async set(key, value, expiration = this.defaultExpiration) {
    await this.redis.setex(key, expiration, JSON.stringify(value));
  }

  async invalidatePattern(pattern) {
    const keys = await this.redis.keys(pattern);
    if (keys.length) {
      await this.redis.del(keys);
    }
  }
}

```

```
}  
}
```

27. Advanced Logging and Monitoring

javascript

 Copy

```
const winston = require('winston');  
const { createLogger, format, transports } = winston;  
const { combine, timestamp, printf, colorize } = format;  
  
const customFormat = printf(({ level, message, timestamp }) => {  
  return `${timestamp} ${level}: ${message}`;  
});  
  
const logger = createLogger({  
  level: 'info',  
  format: combine(  
    colorize(),  
    timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),  
    customFormat  
  ),  
  transports: [  
    new transports.Console(),  
    new transports.File({ filename: 'error.log', level: 'error' }),  
    new transports.File({ filename: 'combined.log' })  
  ]  
});
```

Would you like me to continue with more advanced Node.js implementations?

 Claude does not have the ability to run the code it generates yet.

P continue

Here are advanced Node.js concepts and implementations:

28. Advanced Authentication with OAuth

javascript

 Copy

```
const passport = require('passport');  
const GoogleStrategy = require('passport-google-oauth20').Strategy;
```

```

passport.use(new GoogleStrategy({
  clientId: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: "/auth/google/callback"
}),
async (accessToken, refreshToken, profile, done) => {
  const existingUser = await User.findOne({ googleId: profile.id });

  if (existingUser) {
    return done(null, existingUser);
  }

  const newUser = await new User({
    googleId: profile.id,
    email: profile.emails[0].value
  }).save();

  done(null, newUser);
});

```

29. Advanced Microservices with gRPC

javascript

 Copy

```

const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');

const packageDefinition = protoLoader.loadSync('user.proto');
const userProto = grpc.loadPackageDefinition(packageDefinition).user;

const server = new grpc.Server();
server.addService(userProto.UserService.service, {
  getUser: (call, callback) => {
    const user = {
      id: call.request.id,
      name: 'John Doe',
      email: 'john@example.com'
    };
    callback(null, user);
  }
});

server.bindAsync(
  '0.0.0.0:50051',

```

```
grpc.ServerCredentials.createInsecure(),
() => server.start()
);
```

30. Advanced Pagination Strategy

javascript

 Copy

```
class PaginationService {
  async paginate(model, query = {}, options = {}) {
    const page = parseInt(options.page) || 1;
    const limit = parseInt(options.limit) || 10;
    const skipIndex = (page - 1) * limit;

    const results = await model
      .find(query)
      .sort(options.sort || { createdAt: -1 })
      .limit(limit)
      .skip(skipIndex)
      .exec();

    const total = await model.countDocuments(query);

    return {
      results,
      currentPage: page,
      totalPages: Math.ceil(total / limit),
      totalItems: total
    };
  }
}
```

31. Advanced Pub/Sub with Kafka

javascript

 Copy

```
const { Kafka } = require('kafkajs');

class KafkaService {
  constructor() {
    this.kafka = new Kafka({
      clientId: 'my-app',
      brokers: ['localhost:9092']
    });
    this.producer = this.kafka.producer();
    this.consumer = this.kafka.consumer({ groupId: 'my-group' });
  }
}
```

```

}

async publish(topic, message) {
  await this.producer.connect();
  await this.producer.send({
    topic,
    messages: [{ value: JSON.stringify(message) }]
  });
}

async subscribe(topic, callback) {
  await this.consumer.connect();
  await this.consumer.subscribe({ topic, fromBeginning: true });

  await this.consumer.run({
    eachMessage: async ({ message }) => {
      const value = JSON.parse(message.value.toString());
      callback(value);
    }
  });
}
}

```

32. Advanced Validation with Joi

javascript

 Copy

```

const Joi = require('joi');

const userSchema = Joi.object({
  username: Joi.string()
    .alphanum()
    .min(3)
    .max(30)
    .required(),

  password: Joi.string()
    .pattern(new RegExp('^[a-zA-Z0-9]{3,30}$')),

  email: Joi.string()
    .email({
      minDomainSegments: 2,
      tlds: { allow: ['com', 'net'] }
    })
});

```

```
function validateUser(user) {
  const { error, value } = userSchema.validate(user);

  if (error) {
    throw new Error(error.details[0].message);
  }

  return value;
}
```

33. Advanced Caching with Multi-Level Strategy

javascript

 Copy

```
class MultiLevelCache {
  constructor(redisClient, localCache) {
    this.redis = redisClient;
    this.local = localCache;
  }

  async get(key) {
    // Check local cache first
    let value = this.local.get(key);

    if (value) return value;

    // Then check Redis
    value = await this.redis.get(key);

    if (value) {
      // Populate local cache
      this.local.set(key, JSON.parse(value));
      return JSON.parse(value);
    }

    return null;
  }

  async set(key, value, ttl = 3600) {
    // Set in local cache
    this.local.set(key, value);

    // Set in Redis with expiration
    await this.redis.setex(key, ttl, JSON.stringify(value));
  }
}
```

34. Advanced Rate Limiting with Token Bucket

javascript

 Copy

```
class RateLimiter {
  constructor(capacity, refillRate) {
    this.capacity = capacity;
    this.tokens = capacity;
    this.refillRate = refillRate;
    this.lastRefillTime = Date.now();
  }

  async consume(tokens = 1) {
    this.refill();

    if (this.tokens >= tokens) {
      this.tokens -= tokens;
      return true;
    }

    return false;
  }

  refill() {
    const now = Date.now();
    const timePassed = now - this.lastRefillTime;
    const tokensToAdd = (timePassed / 1000) * this.refillRate;

    this.tokens = Math.min(
      this.capacity,
      this.tokens + tokensToAdd
    );

    this.lastRefillTime = now;
  }
}
```

Would you like me to elaborate on any of these advanced Node.js concepts?

① Claude does not have the ability to run the code it generates yet.

P continue

Here are advanced Node.js concepts continued:

35. Advanced Error Handling Patterns

javascript

 Copy

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

const errorHandler = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;
  err.status = err.status || 'error';

  res.status(err.statusCode).json({
    status: err.status,
    message: err.message,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
};
```

36. Advanced Dependency Injection Container

javascript

 Copy

```
class DIContainer {
  constructor() {
    this.dependencies = new Map();
  }

  register(key, dependency) {
    this.dependencies.set(key, dependency);
  }

  resolve(key) {
    const dependency = this.dependencies.get(key);
    if (!dependency) {
      throw new Error(`Dependency ${key} not found`);
    }
    return dependency;
  }
}
```

```
factory(key, factory) {
  this.dependencies.set(key, { factory });
}
}
```

37. Advanced Middleware Composition

javascript

 Copy

```
const compose = (...middlewares) => {
  return middlewares.reduce((a, b) => async (ctx, next) => {
    await a(ctx, async () => {
      await b(ctx, next);
    });
  });
};

const logger = async (ctx, next) => {
  console.log('Logger middleware');
  await next();
};

const authentication = async (ctx, next) => {
  console.log('Authentication middleware');
  await next();
};

const composedMiddleware = compose(logger, authentication);
```

38. Advanced Scheduling System

javascript

 Copy

```
const schedule = require('node-schedule');

class SchedulerService {
  scheduleJob(pattern, callback) {
    return schedule.scheduleJob(pattern, callback);
  }

  cancelJob(job) {
    job.cancel();
  }

  createRecurringTask(interval, callback) {
```

```
    return setInterval(callback, interval);
  }
}
```

39. Advanced Performance Profiling

javascript

 Copy

```
const v8 = require('v8');

class PerformanceProfiler {
  static takeHeapSnapshot() {
    const snapshot = v8.getHeapSnapshot();
    const fileName = `heap-${Date.now()}.heapsnapshot`;

    return new Promise((resolve, reject) => {
      const writeStream = require('fs').createWriteStream(fileName);
      snapshot.pipe(writeStream);

      writeStream.on('finish', () => resolve(fileName));
      writeStream.on('error', reject);
    });
  }

  static measureExecutionTime(fn) {
    const start = process.hrtime();
    const result = fn();
    const [seconds, nanoseconds] = process.hrtime(start);

    return {
      result,
      executionTime: `${seconds}.${nanoseconds} seconds`
    };
  }
}
```

40. Advanced Authorization Decorator

javascript

 Copy

```
function authorize(roles = []) {
  return (target, key, descriptor) => {
    const originalMethod = descriptor.value;

    descriptor.value = async function(...args) {
      const [req] = args;
```

```

    if (!req.user) {
      throw new Error('Unauthorized');
    }

    if (roles.length && !roles.includes(req.user.role)) {
      throw new Error('Insufficient permissions');
    }

    return originalMethod.apply(this, args);
  };

  return descriptor;
};
}

class UserController {
  @authorize(['admin'])
  async deleteUser(req, res) {
    // Delete user logic
  }
}

```

41. Advanced Retry Mechanism

javascript

 Copy

```

class RetryService {
  static async retry(
    fn,
    {
      maxAttempts = 3,
      delay = 1000,
      backoff = 2
    } = {}
  ) {
    let attempts = 0;

    while (attempts < maxAttempts) {
      try {
        return await fn();
      } catch (error) {
        attempts++;
      }

      if (attempts >= maxAttempts) {
        throw error;
      }
    }
  }
}

```

```
    }

    await new Promise(resolve =>
      setTimeout(resolve, delay * Math.pow(backoff, attempts))
    );
  }
}
}
```

Would you like me to continue with more advanced Node.js implementations?

📘 Claude does not have the ability to run the code it generates yet.

📄 Copy ↺ Retry 👍 🗨



Claude can make mistakes. Please double-check responses.